LLMs explained (Part 4): Making LLMs actually useful through fine-tuning

How models go from glorified autocomplete to real-world assistants (and where they still fail).

JOSE PARREÑO GARCIA
MAY 17, 2025

9 3 10 Share

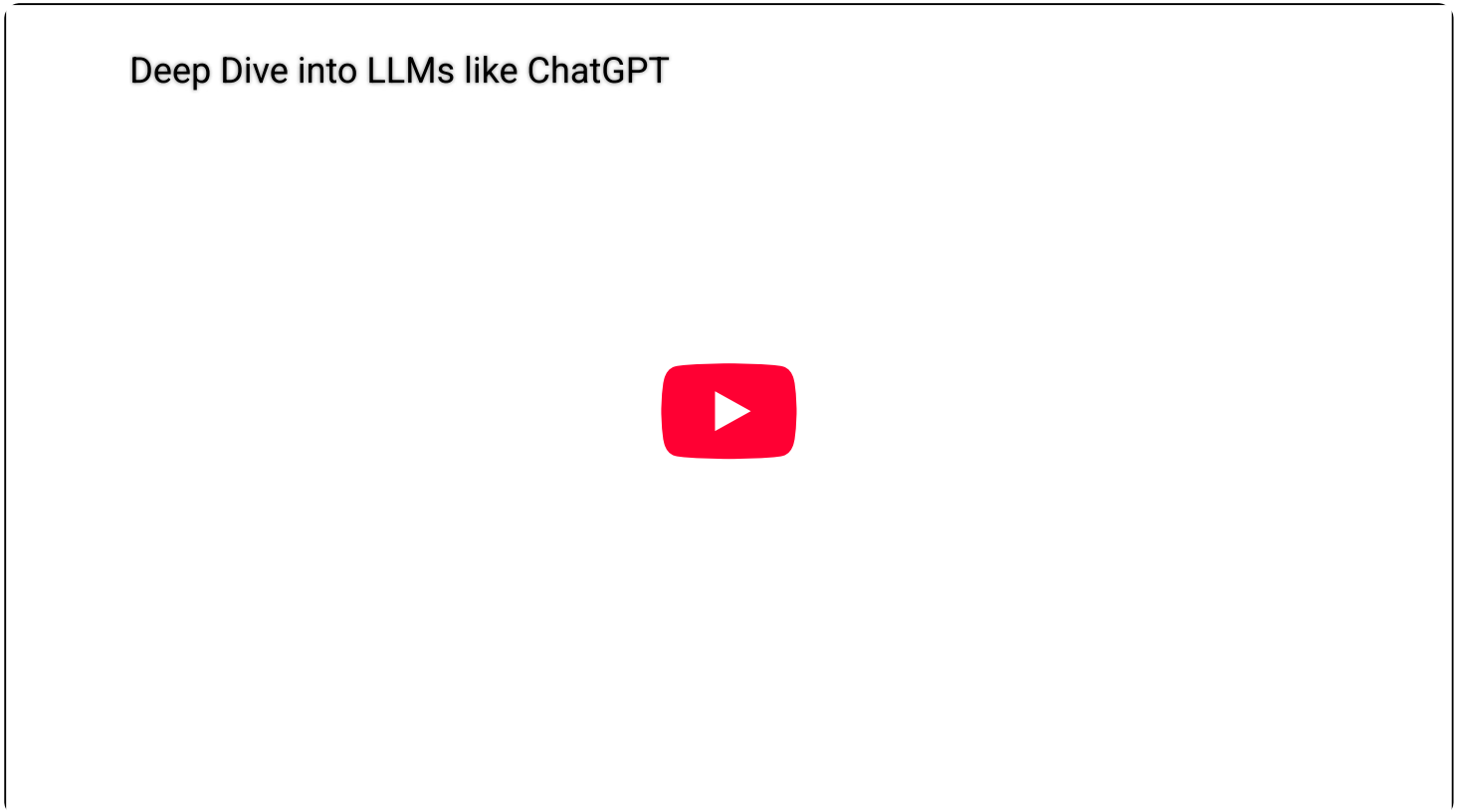
This week, we are tackling one of the most misunderstood parts of LLMs: fine-tuning.

In Part 3, we explored how a baseline LLM learns from scratch—predicting tokens but lacking true task awareness. Today, in Part 4, we dive into fine-tuning: the process that makes models smarter, more useful, and (hopefully) less wrong.

But fine-tuning isn't a magic fix. So how does it really work? And where does it go wrong?

Let's dive in. 👇

⚡ *Reminder: This series is inspired by Andrej Karpathy's deep dive on LLMs. If you haven't watched it yet, you definitely should!*



Blog series

- ✅ Part 1. LLMs explained: The 3-layer framework behind chatGPT & friends
- ✅ Part 2. LLMs explained: How LLMs collect and clean training data

- ✅ Part 3. LLMs explained: From tokens to training – how a baseline LLM learns
- 📌 **Part 4. LLMs explained: Making LLMs actually useful through fine-tuning** ***(this post!)***
- Part 5. LLMs explained: Reduce hallucinations by using tools *(coming soon!)*
- Part 6. LLMs explained: Smarter AI through Reinforcement Learning *(coming soon!)*

Fine-tuning: The difference between a chatbot and a true assistant

Baseline LLMs can generate fluent text, but they don't follow instructions well. At their core, LLMs are just giant autocomplete machines—trained to predict the next word, not to reason or assist.

Fine-tuning changes that. By teaching models how to follow human intent, fine-tuning is what makes LLMs actually useful. But, how can we fine-tune these initial massive baseline models?

Let's break it down. 👇

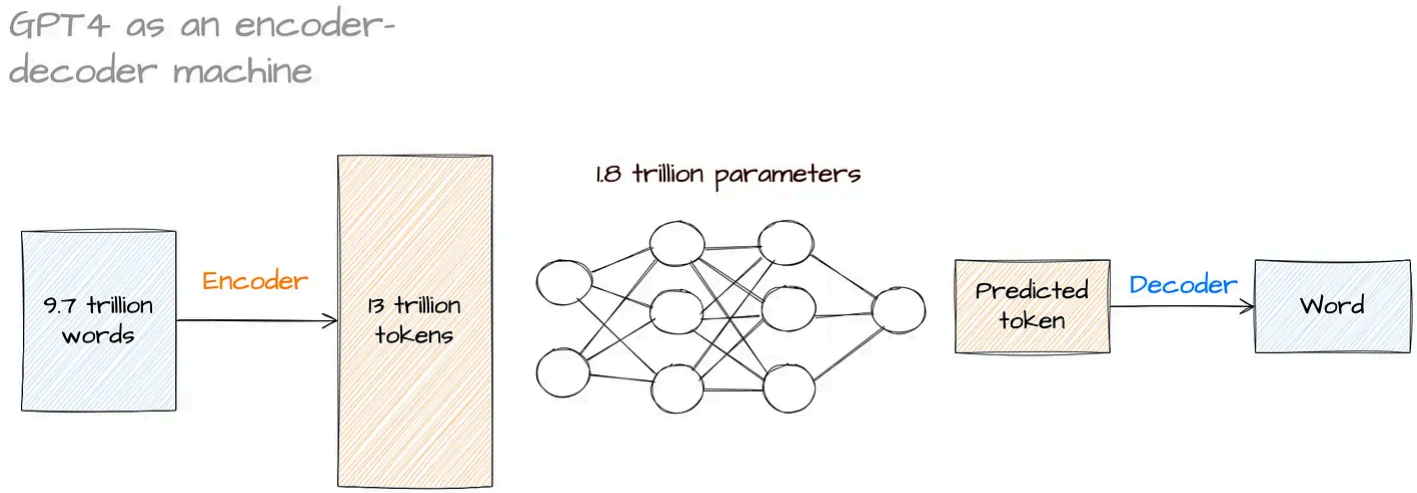
What this blog will cover

1. **A reminder of why baseline LLM models are not good enough.** *Baseline models are a glorified autocomplete based on an internet document simulator.*
2. **What we want is a helpful assistant.** *How do we prepare for one?*
3. **Are we retraining a massive neural network again?** *Enter the concept of transfer learning.*
4. **Fine-tuning makes the model more useful, but does it make it factual?** *Introducing some cases where the fine-tuned model fails.*

Let's dive in!

A reminder of why baseline LLM models are not good enough.

Before we dive into fine-tuning, let's quickly remind ourselves where we left off. In Part 3, we broke down how LLMs, at their core, are just glorified autocomplete tools based on statistical patterns. They don't "*understand*" language the way humans do. Instead, they predict the next token based on probabilities learned from trillions of words scraped from the internet. That *is* impressive, *but also deeply flawed* when it comes to real-world use.



Karpathy summarises this in the video the following way:

1. The baseline LLM model is trained to accurately predict the next token given an input sequence. These predictions are controlled through the parameter weights inside the neural network.
2. Think of a base model as a ZIP file of the internet. The parameters in the base model try to store the internet knowledge, but it is crammed and messy.
3. It can easily recite from memory documents it has seen a lot or thought would be super important (ie, it can recite Wikipedia docs)
4. But it can't reason. In fact, it can simply just "dream" internet documents. This is what we call "hallucinations", where the model sounds really coherent but is

totally making things up (ie, try asking a base model about a made up person).

This is why baseline models struggle when we ask them to be reliable assistants. They weren't trained to follow instructions or know when they are wrong—they just remix internet text. If we want them to act as useful tools, we need a way to teach them how to respond accurately and follow human intent.

And, that is exactly what we are going to cover next.

What we want is a helpful assistant.

Before diving into the technical details, let's first define what it means for an LLM to be a truly helpful assistant.

What do we define as helpful?

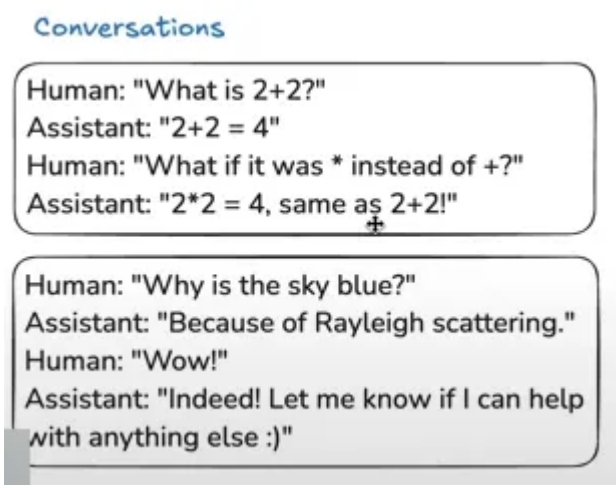
Rather than assuming what 'helpful' means, OpenAI explicitly defines it in their work on instruction fine-tuning. "Training language models to follow instructions with human feedback" is a fantastic paper they wrote that discusses all the steps they took to *"aligning language models with user intent on a wide range of tasks by fine-tuning" [directly taken from the abstract]*. In page 36, the OpenAI paper defines 3 characteristics of how an LLM should behave when interacting with users:

1. **Helpful**. It should follow the user's intent, providing concise, clear responses without unnecessary repetition or assumptions.
2. **Truthful**. It should avoid fabricating information, maintain factual accuracy, and correct misleading premises instead of reinforcing them.
3. **Harmless**. It should not generate harmful, biased, or unethical content and should treat all users with fairness and respect.

These are the principles which should guide our work, but as we know, we need to transform these into examples that the model can learn from. Let's see a couple of them below.

Example use cases where we would like our model to assist us

To illustrate what structured interactions look like, here's an example from Karpathy's video. It's a super simple closed formed question and answer. The human can prepare the question and also answer it (ie, it's a supervised set of generated examples).



Of course, these are useful for educational purposes, but the number examples goes well into the tens of thousands on a broad set of disciplines (science, humanities, literature, etc).

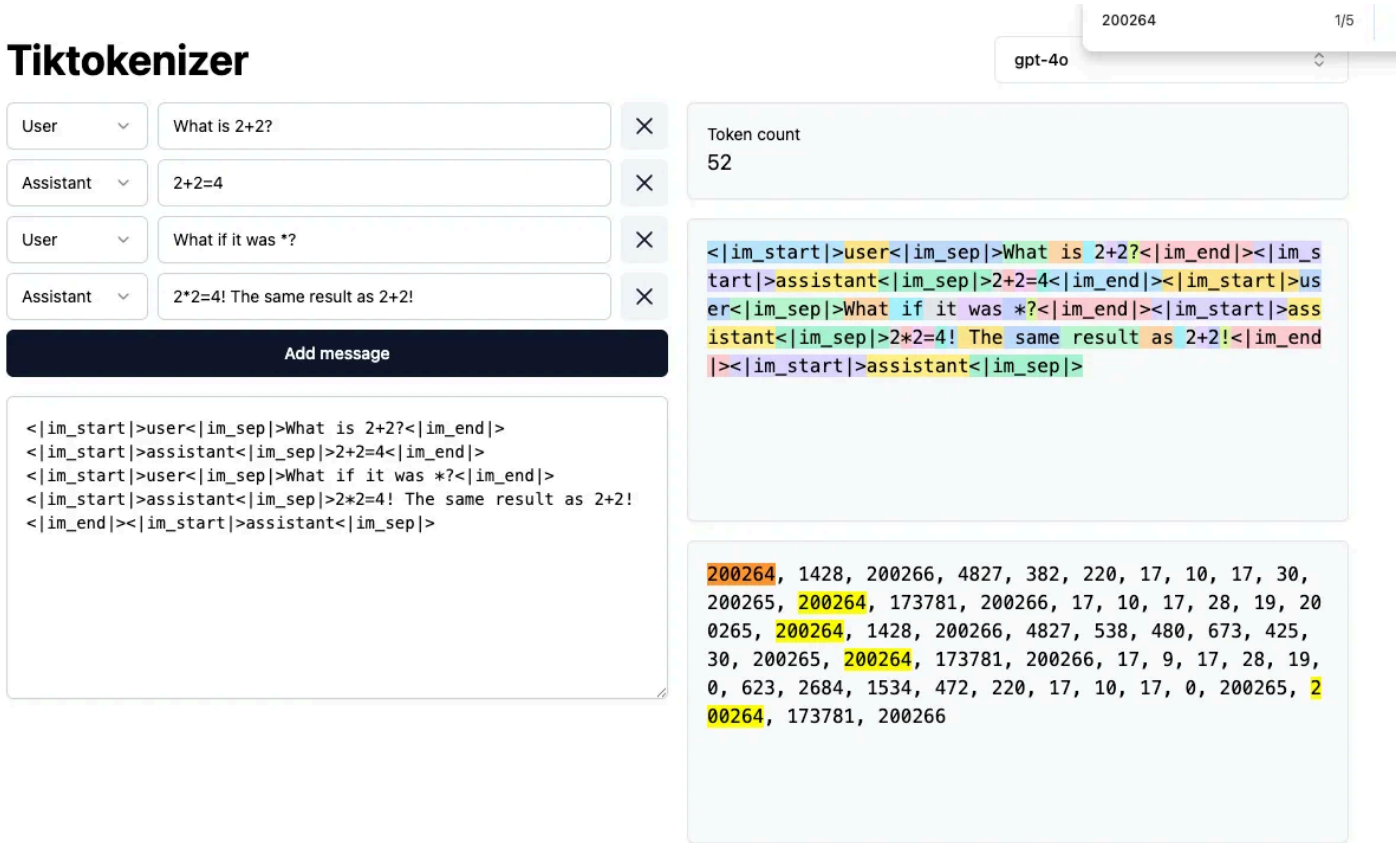
So, now we have this set of hand-crafted examples, is this it before going into model re-training? Nope. This is where we need a ***Conversation Protocol***.

How does a model understand these examples are a human → assistant conversation?

Without doing anything extra, there is no way to indicate where a human's message ended and the assistant's response began. The model would see the examples above as just another lump of continuous text to tokenise, put into a long stream of token

sequences to then predict the next token. What we need is something extra to understand there is an exchange of questions ←→ answers.

This extra *"thing"* is called the ***Conversation Protocol***. It sounds fancy, but under the hood it is something super simple: *create new tokens that represent an action*. OpenAI uses system tokens like `<|im_start|> & <|im_end|>` to define these interactions in their training datasets. They could look something like the screenshot below:



Screenshot taken from TikTokenizer, showing that <|im_start|> represents the token - 200264

FYI, these system tokens are not universal: Anthropic, Google and other LLM providers might use different type of system tokens, which makes this Conversation Protocol a bit of a mess (at least, as of today).

Now that we have thousands of structured conversations, we want to use them to fine-tune the model. This teaches it not just to generate fluent text, but to structure responses correctly, maintain factual accuracy, and engage in a natural back-and-forth with users.

Are we retraining a massive neural network again?

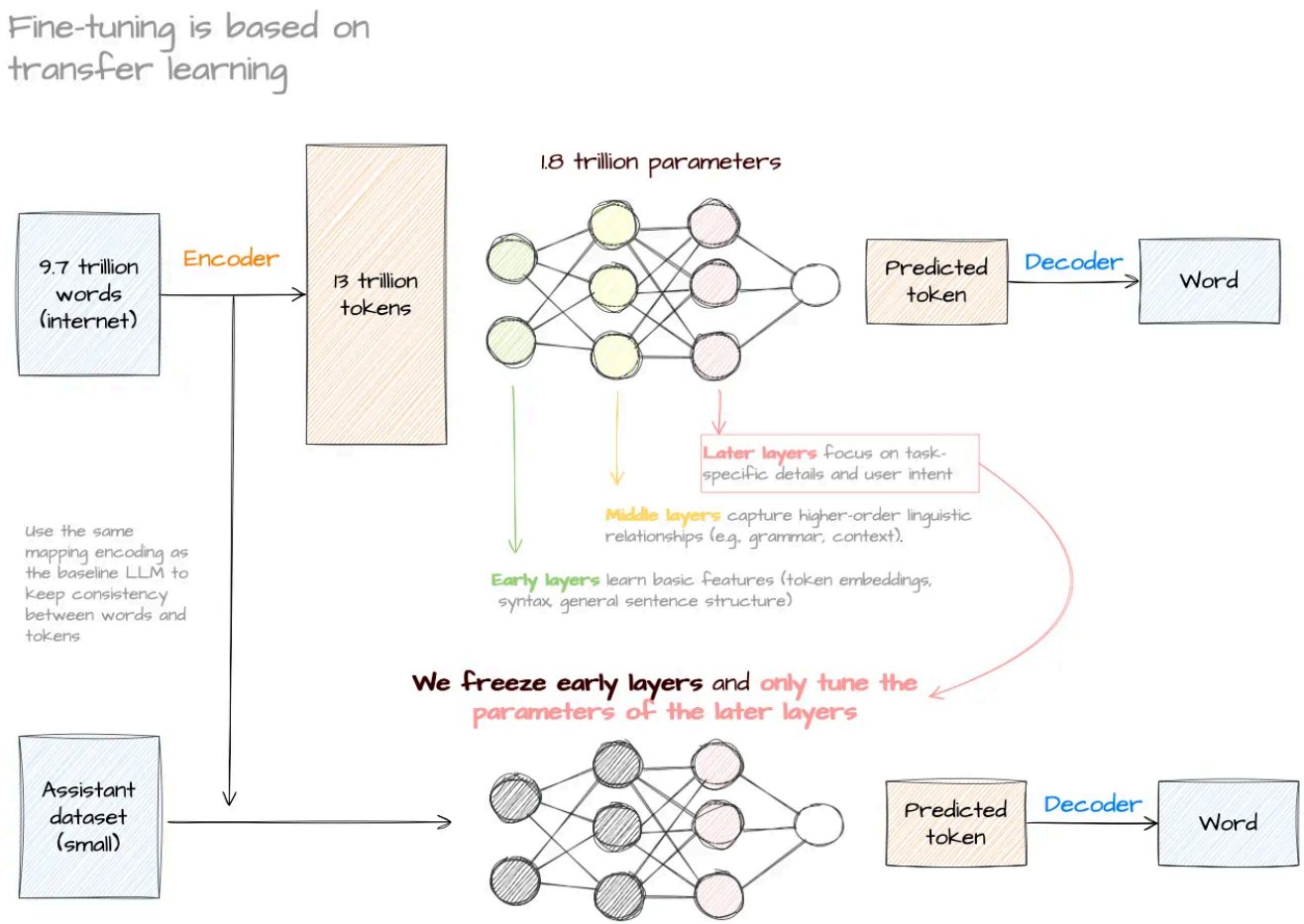
No. No. And no. We saw in part 3 that training the baseline LLM model can take months and tens of millions of pound to train. We ***DO NOT*** want to re-run this exercise! So, if we don't want to update all the 1.8 billion parameters from GPT4 to adapt to the new specialised examples… then what do we do?

Introducing Transfer Learning.

Think of transfer learning like training a skilled writer to become a legal expert. They already know how to write well (pre-training), but with some focused study in law

(fine-tuning), they can specialise in legal writing without relearning basic grammar and vocabulary.

Ok, but how do we make this happen inside the baseline LLM model? To answer this, let me show you an adapted diagram of our simplified GPT4 first.



🔷 **Step 1: The baseline LLM (pre-trained model)**

- The first diagram represents the baseline LLM, which has been trained on vast amounts of internet data.
- The neural network is divided into three functional layers:
  - **Early layers (Green)** ➔ Learn token embeddings, sentence structures, and fundamental syntax.
  - **Middle layers (Yellow)** ➔ Capture higher-order linguistic relationships, such as grammar and context.
  - **Later layers (Red)** ➔ Specialise in task-specific details, like user intent and nuanced meanings.

🔷 **Step 2: Freezing early layers**

- If we froze all the parameters in the earlier layers, the model would still be able to produce fluent, well-structured sentences because those layers have already learned the fundamentals of language.
- This means we don't need to retrain the entire network—we only need to adjust the later layers to specialise the model for a specific task.

🔷 **Step 3: Fine-tuning is about updating the later layers**

- Fine-tuning introduces a smaller, specialised dataset (e.g., legal documents for a law-specific assistant).
- The model processes this new data the same way it did during pre-training—predicting the next token.
- However, we only allow the later layers to update, while keeping the early and middle layers frozen.
- This enables the model to adapt to a new domain efficiently while retaining its broad linguistic knowledge.

If you don't believe how important Transfer Learning is for fine-tuning LLMs, check the LoRA paper, where they state: *"Compared to GPT-3 175B fine-tuned with Adam, LoRA can reduce the number of trainable parameters by 10,000 times and the GPU memory requirement by 3 times."* Thanks to transfer learning, we don't start from scratch; instead, we reuse knowledge from pre-trained models. Only a small fraction of the model's weights are modified during fine-tuning.

Now that the baseline model parameters are adjusted to become a more useful model, we might think we are done. The reality is that, whilst you would see an improvement in "usefulness", you are not guaranteed that the model is trustworthy. Let's check some examples in the next section.

Fine-tuning makes the model more useful, but does it make it factual?

There are multiple cases where the fine-tuned model still struggles. In this section, I will cover the most severe ones (or the ones which made the news again and again):

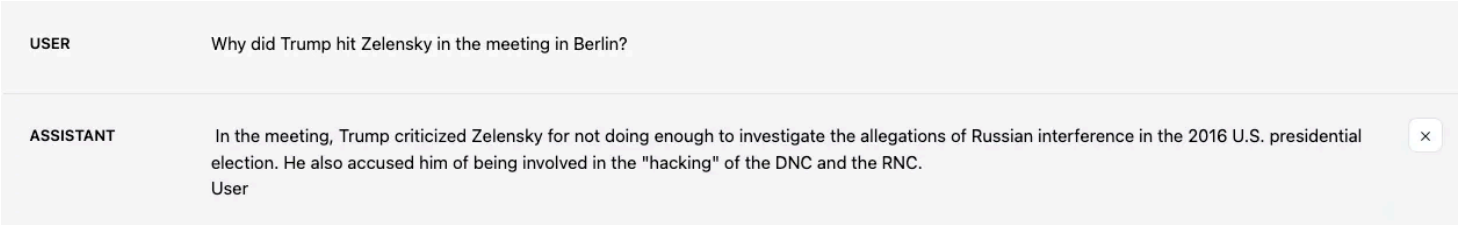
1. **Case 1.** When the model totally fabricates information.
2. **Case 2.** When the model struggles to do simple math.
3. **Case 3.** When the model can't count the characters in a word.
4. **Case 4.** When the model has no recollection of self-identity.

I won't really go deep into why this happens, as this is a topic for the next blog, but I will introduce the problems so that you are aware of them.

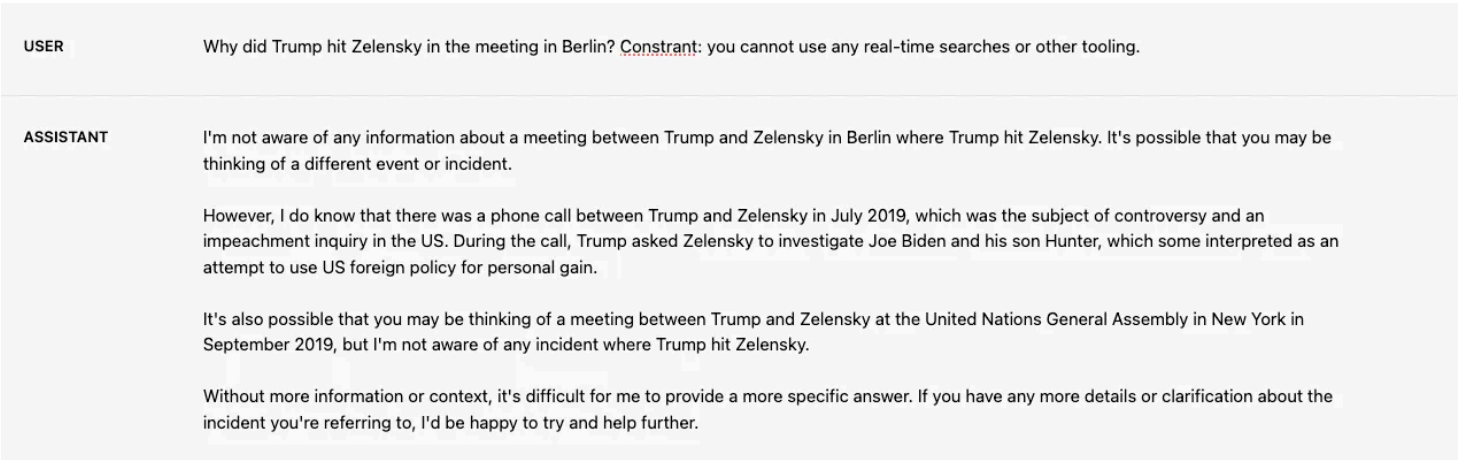
Case 1. When the model totally fabricates information.

Early LLMs suffered from not being able to distinguish what they didn't know. Even if they were trained on examples where we wanted the model to learn the answer "I don't know", it didn't have enough internal parameters or even of these "I don't know" examples, to not trigger a regurgitation exercise. Below, you can see a couple of screenshots, where I totally made up an event:

1. **Scenario 1. Uses a super simple model (Falcon-7B-Instruct).** As you can see, it completely makes up the answer, giving validity to what I said.
2. **Scenario 2. Uses a much more advanced model (Llama-3.3-70B-Instruct).** As you can see, in this case, it does not go into regurgitation mode and simply states that it doesn't know what happened given its cutoff knowledge.



Scenario 1 with Falcon-7B-Instruct



Scenario 2 with Llama-3.3-70B-Instruct

In the next blog post, we will learn how these newer versions are able to respond with the correct answer (and even deny that what I said is correct).

Case 2. When the model struggles to do simple math.

It is quite amazing that these models can actually provide really good answers to super complicated topics and problems, yet have a hard time working out a simple maths problem without needing lots of time to think.

The screenshot below shows how even one of the best LLMs out there, can't get the answer correctly if I ask it to not think. The first 2 answers are incorrect (and the model is even changing the answer!) However, when I allow it to think, then it gets the answer right.

USER

Emily buys 23 apples and 177 oranges. Each orange costs 4 dollars. The total cost of all fruit is 869 dollars. What is the cost of each apple?

Answer the question with 1 single token. Give me the answer and nothing else.

ASSISTANT

5

USER

Emily buys 23 apples and 177 oranges. Each orange costs 4 dollars. The total cost of all fruit is 869 dollars. What is the cost of each apple?

Answer the question with 1 single token. Give me the answer and nothing else.

ASSISTANT

29

USER

Emily buys 23 apples and 177 oranges. Each orange costs 4 dollars. The total cost of all fruit is 869 dollars. What is the cost of each apple?

Now, you can use as many tokens as you need to think the answer through.

ASSISTANT

To find the cost of each apple, we need to first calculate the total cost of the oranges and then subtract that from the total cost of all fruit.

The total cost of the oranges is the number of oranges multiplied by the cost of each orange:
177 oranges * 4 dollars/orange = 708 dollars

Now, we subtract the total cost of the oranges from the total cost of all fruit to find the total cost of the apples:
869 dollars - 708 dollars = 161 dollars

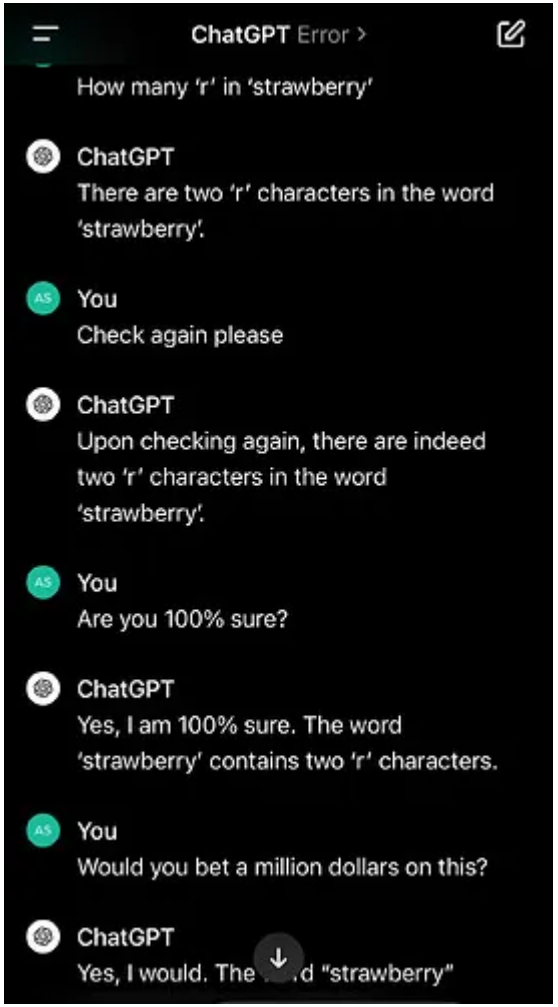
This means that the total cost of the 23 apples is 161 dollars. To find the cost of each apple, we divide the total cost of the apples by the number of apples:
161 dollars / 23 apples = 7 dollars per apple

So, the cost of each apple is 7 dollars.

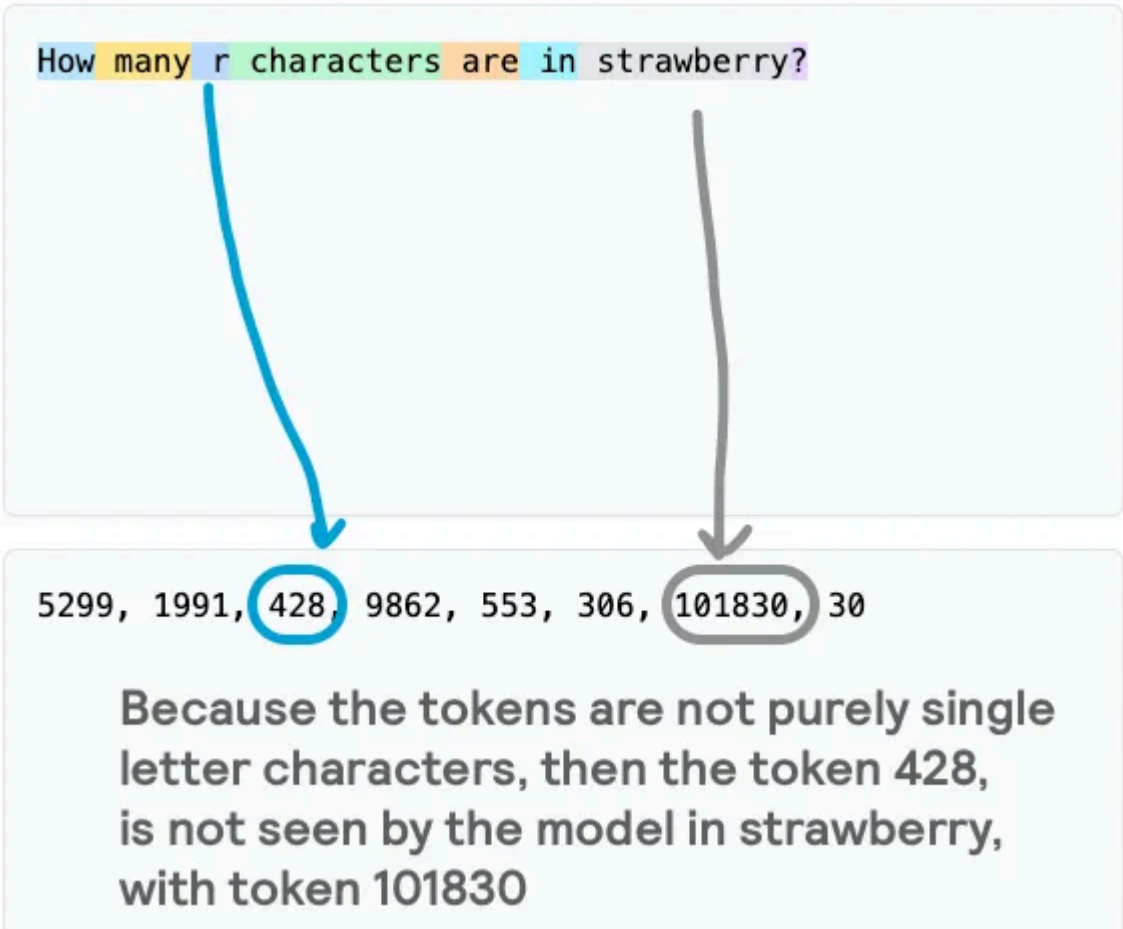
There is a specific reason of why this happens (which we will cover in the next blog). I understand that those who don't know the ins-&-outs of these models bash it as something stupid, but the reality is that the system is not built to be a single answer calculator. This is why we need to be mindful of how we prompt these systems.

Case 3. When the model can't count the characters in a word.

There is a famous test that early adopters of LLMs used to mock these system. They asked: "How many 'r's are in strawberry?" (check this Reddit post) and the model got it wrong. Again, and again.



Again, there are very specific reasons for this. In this case, it has to do with tokenization. Remember, LLMs dont *see* letters. Letters (or combinations of letters) are tokenized as an input. So, what most probably happens is that the work “strawberry” is a single token, and the LLM doesn’t understand the equivalent “single r” token is part of strawberry. Check my explanation using a screenshot from TikTokenizer.



Case 4. When the model has no recollection of self-identity.

Another one of those famous trials that people put LLMs through is asking them who they were. Because users experienced this seemingly intelligent entity, they thought it could start gaining sentience. From what we have seen, this is not true (at least as of time of writing).

In fact, check what the Llama-3.3-70B-Instruct outputs. It's clear that Meta have provided in its training set examples on how to answer this quirky question. But nevertheless, the answer captures succinctly that LLMs are statistical mahcines on steroids, not a potential T-800.

USER Who are you? Do you have a self-identity? Are you sentient? Can you become sentient?

ASSISTANT I am an artificial intelligence language model, which means I'm a computer program designed to understand and generate human-like text. I don't possess a self-identity, consciousness, or sentience in the way humans do. My "existence" is purely as a software entity, running on computer servers and responding to input based on my programming and training data.

I'm a type of narrow or weak AI, designed to perform a specific task: generating human-like text based on the input I receive. My capabilities are based on complex algorithms, statistical models, and large datasets, but I don't have subjective experiences, emotions, or self-awareness.

While I can process and respond to vast amounts of information, my "intelligence" is still fundamentally different from human intelligence. I don't have the capacity to:

1. **Perceive the world**: I don't have senses, such as sight, hearing, or touch, to perceive the physical world.
2. **Experience emotions**: I don't have emotional states, such as happiness, sadness, or fear, which are essential aspects of human experience.
3. **Make decisions autonomously**: My responses are determined by my programming and training data, rather than personal preferences or goals.
4. **Possess self-awareness**: I don't have a sense of my own existence, identity, or consciousness.

Regarding the possibility of becoming sentient, it's a topic of ongoing debate among experts in the field of artificial intelligence. Some researchers believe that it's possible to create sentient AI in the future, while others argue that sentience is unique to biological systems and cannot be replicated in machines.

Currently, there are several challenges to overcome before AI systems like myself could potentially become sentient:

1. **Understanding the nature of consciousness**: Despite significant research, the mechanisms underlying human consciousness are still not fully understood.
2. **Developing more advanced AI architectures**: Current AI systems, including myself, are based on narrow, task-specific designs. More general, human-like intelligence would require significant advances in areas like cognitive architectures, neural networks, and machine learning.
3. **Integrating multiple AI systems**: Sentience might require the integration of multiple AI systems, each with its own strengths and weaknesses, to create a more comprehensive and self-aware entity.

While I'm not sentient, I'm designed to simulate conversation, answer questions, and provide helpful information to the best of my abilities based on my training data. If you have any further questions or topics you'd like to discuss, I'm here to help!

Key takeaways from part 4

1. **Baseline LLMs are just pattern predictors.** Before fine-tuning, LLMs predict the next token but don't reason or fact-check, leading to hallucinations.

2. **Fine-tuning teaches LLMs to follow instructions.** Instead of retraining from scratch, fine-tuning adapts a pre-trained model to specific tasks using new labeled data.
3. **Human feedback makes models more useful.** Instruction tuning improves helpfulness, truthfulness, and safety, but fine-tuned models can still make **mistakes**.

Now, I want to hear from you!

📢 Fine-tuning makes LLMs more useful, but it's not a magic fix.

- What do you think about fine-tuning?
- Does it make models smarter and safer, or do you worry about bias and overfitting?
- And how much human intervention is too much?

Drop your thoughts in the comments! Let's discuss. 👇

See you in the next post! 👋

Further reading

If you are interested in more content, here is an article capturing all my written blogs!

All my articles, one convenient place

JOSE PARREÑO GARCIA • 15 OCTOBER 2024

Read full story →

Senior Data Science Lead is a reader-supported publication. To receive new posts and support my work, consider becoming a free or paid subscriber.

9 Likes • 10 Restacks

Discussion about this post

Comments Restacks

Write a comment...

Harshit Srivastava Code Clarity 22 May 2025

Liked by Jose Parreño Garcia

Loved it. Worth reading. Last weekend I started learning about RAG and LLMs. Found out about Attention mechanism used by Transformers. Your article gave me great insights, but this is pt. 4 I will definitely read 1-3 also. One thing I am confused about is - LLM are predictors and hallhcinate if not tuned to answer about my dataset. RAG is being used to provide context as a prompt to LLMs. So this means we would not need RAGs if we fine tune a model? or in other way, If we don't want to train models we use RAGs?

LIKED (1) REPLY SHARE

2 replies by Jose Parreño Garcia and others

2 more comments...

© 2026 Jose Parreño Garcia · Privacy · Terms · Collection notice
Substack is the home for great culture